SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



4. C Übersetzung

Aufteilung des Quelltexts

- Deklarationen globaler Objekte[1] in Header-Datei (`*.h`)
  - eigene Datentypen
  - globale Variablen
  - “öffentliche” Funktionen

- Definitionen und privaten Objekte in Quelltext-Datei (`*.c`)
  - ev. Deklaration von privaten (`static`) Objekten
  - Eigener Header wird mit Präprozessor *inkludiert*
    - überprüft ob Deklaration und Definition übereinstimmen

[1] mit *“Objekte”* sind in C Typen, Variablen oder Funktionen gemeint

```c
// file: my.h
extern int global_variable;
extern int global_function(int a, int b);
```

Zusatz `extern` ist implizit und kann weggelassen werden

```c
// file: my.c
#include "my.h" // NOTE: own header first
#include <stdio.h> // NOTE: other headers

int global_variable = 0; // definition
static void private_function(int a); // declaration

int global_function(int a, int b) {
  /* implementation ... */
}

void private_function(int a) {
  /* implementation ... */
}
```

Übersetzungsvorgang – Überblick (i)

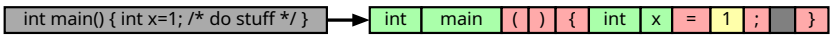
- 1. Phase: Einlesen des Quelltextes
- 2. Phase: geschützte Zeilenumbrüche entfernen

```
prin\
tf("bro\
ken?");
```

→

```
printf("broken?");
```

- 3. Phase: Tokenisierung
  - erkennen von Bezeichnern, Literalen, Operatoren, …
  - Kommentare werden durch einzelnes Leerzeichen ersetzt



Übersetzungsvorgang – Überblick (ii)

- 4. Phase: Präprozessor führt Textersetzung durch
- 5. Phase: Umwandeln von *source* in *execution character set*
- 6. Phase: Zeichenketten verknüpfen

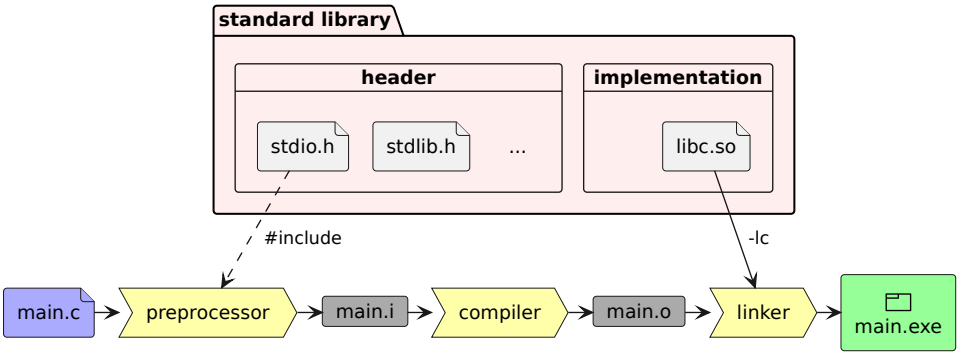
```
printf("This" "is" "a" "single" "string"); // -> Thisisasinglestring
```

- 7. Phase: Übersetzung in Maschinensprache (sogenannter »*Objektcode*«)
  - tatsächliche Übersetzung (*compilation*)
- 8. Phase: Binden (*linking*)
  - Objektcode von mehreren Dateien wird verknüpft
  - Objektcode von Bibliotheken wird verknüpft
  - Ergebnis ist Speicherabbild (*image*)
    - ausführbare Datei (*executable*)
    - Bibliothek (*library*)

Phases of translation - cppreference.com

Übersetzungsvorgang Schematisch

- Einbinden von Headern mit `#include` Anweisung
  - werden zuerst hinein “kopiert” und dann selbst vorverarbeitet
- Binden von Bibliotheken mit Compiler-Parameter (`-l`)
  - enthalten Implementierung von vorhandenen Funktionen
  - z.B. `-lc` bindet Standardbibliothek (`libc.so`) ein[1]



[1] Binden mit der Standardbibliothek mit `-lc` passiert meist automatisch

Präprozessor (i)

- einfache (stupide) Textersetzung

wichtigste Präprozessor Direktiven

- `#include`
  - “Kopieren” des Inhalts von anderen Dateien, Einbinden von Deklarationen
  - spitze Klammern für externe Header, z.B. `<stdio.h>`
  - doppelte Hochkommata für eigene Header, z.B. `"utils/myutils.h"`
    - sucht nach Header-Datei auch im aktuellen Verzeichnis
- `#define` und `#undef`
  - Definition von Makros (auch mit Parametern)
- `#if`, `#ifdef`, `#ifndef`, `#else`, `#elif` und `#endif` für bedingte Übersetzung
  - ermöglicht einzelne Abschnitt nur unter bestimmten Voraussetzungen zu übersetzen
- `#pragma` für Compiler-Interaktion
  - z.B. deaktivieren von Warnungen für bestimmte Zeilen
- `#line` um Dateiname und Zeile anzugeben, z.B. für generierten Quelltext

Präprozessor: Konstanten und Makros (i)

- Achtung: kein Strichpunkt am Ende
- Konstanten

```c
#define PI 3.1415
#define NULL ((void*)0)

SHOW(PI);
SHOW(NULL);
```

```
PI = 3.1415
NULL = 0x0
```

Präprozessor: Konstanten und Makros (ii)

- Makros mit Parametern

```c
#define ARRAY_LEN(X) (sizeof(X) / sizeof((X)[0]))

int a[] = {1, 2, 3, 4};
int b[] = {1, 2, 3};

SHOW(ARRAY_LEN(a));
SHOW(ARRAY_LEN(b));
```

```
ARRAY_LEN(a) = 4
ARRAY_LEN(b) = 3
```

```c
#define REPEAT(X) (X); (X); (X); (X); (X);

REPEAT(printf("!"));
int x = 0;
REPEAT(x++);
SHOW(x);
```

```
!!!!!x = 5
```

Präprozessor: Konstanten und Makros (iii)

- mit Hilfe von »#« + Parametername, Zugriff auf Parameter als Text möglich

```c
#define SHOW_INT(X) printf("%s -> %d\n", #X, (X))

SHOW_INT(5);

int five = 5;
SHOW_INT(five);

SHOW_INT(five * 10);

SHOW_INT(3 * 2 + 10 - 21 / 123);
```

```
5 -> 5
five -> 5
five * 10 -> 50
3 * 2 + 10 - 21 / 123 -> 16
```

Präprozessor: PASCAL → C

PASCAL-Code in gültigen C-Code umwandeln

```
// file: preprocessor-pascal.c
#define PROCEDURE int
#define FOR for (size_t
#define TO ; i <
#define DO ; i++)
#define BEGIN {
#define END }
#define PRINT(X) printf("%zu", (X));

#include <stdio.h>

PROCEDURE main() BEGIN
  FOR i = 0 TO 10 DO BEGIN
    PRINT(i)
  END
END
```

```
0123456789
```

Compiler-Switch `-E` führt nur Präprozessor aus

```
gcc -E preprocessor-pascal.c
```

```
# 0 "preprocessor-pascal.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3
# 0 "<command-line>" 2
# 1 "preprocessor-pascal.c"
# 10 "preprocessor-pascal.c"
# 1 "/usr/include/stdio.h" 1 3
// [... contents of stdio.h ... ]
extern int printf (const char *__restrict __format, ...);
// [...]
int main() {
  for (size_t i = 0 ; i < 10 ; i++) {
    printf("%zu", (i));
  }
}
```

Präprozessor: Vordefinierte Symbole

- Bedingte Übersetzung mit `#ifdef`
- vom Compiler vordefinierte Werte einsetzen
  - wie z.B. `__STDC_VERSION__`

```
#ifdef __WIN32
  printf("compiled on Windows");
#  elifdef __APPLE__
  printf("compiled on MacOS");
#  elif defined(__linux__) || defined(__unix__)
  printf("compiled on Linux/UNIX");
#  else
#    error unkown operating system
#endif
printf(" at %s %s\n", __DATE__, __TIME__);
printf("C standard %ld\n", __STDC_VERSION__);
```

```
compiled on Linux/UNIX at Sep 14 2026 14:48:43
C standard 202311
```

- Include-Guards vermeiden Redundanz

```
// file: graph.h
#ifndef GRAPH_H
#define GRAPH_H
typedef struct { ... } graph_t;
#endif
```

```
// file: algorithms.h
#ifndef ALGORITHM_H
#define ALGORITHM_H
#include "graph.h"
graph_t new_graph(size_t n_edges);
#endif
```

```
// file: main.c
#include "graph.h"
#include "algorithm.h" // graph.h included only once

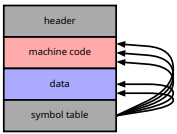
int main() { ... }
```

Compiler (i)

- Compiler übersetzt Resultat des Präprozessors in Maschinensprache
- kann durch Kommandozeilenparameter konfiguriert werden
- Beispiel: Übersetzen einer C-Datei `demo.c` in Objekt-Datei mit GCC

```
gcc -c demo.c
```

- Ergebnis ist Objektdatei `demo.o`
  - Maschinensprache der Funktionen
  - Initialwerte für globalen Variablen
  - Symboltabelle mit Adressen von
    - Funktionen
    - Variablen



Compiler (ii)

```c
// file: demo.c

extern int extern_variable;
extern int extern_function();

int my_variable = {};
void my_function(int x) {
  extern_function();
  extern_variable = 0;
}
static int my_private_function() {
  static int my_private_global = {};
}

int main() {}
```

- Analyse der erzeugten Objektdatei
  - z.B. mit `readelf`, `nm` oder `objdump`

```
nm demo.o
```

```
                 U extern_function
                 U extern_variable
0000000000000024 T main
0000000000000000 T my_function
000000000000001d t my_private_function
0000000000000004 b my_private_global.0
0000000000000000 B my_variable
```

- U ... undefiniert
- T ... im Text-Segment (Maschinensprache)
- B ... im BSS-Segment[1] (globale Variablen)
- Kleinbuchstabe ... lokale Definition

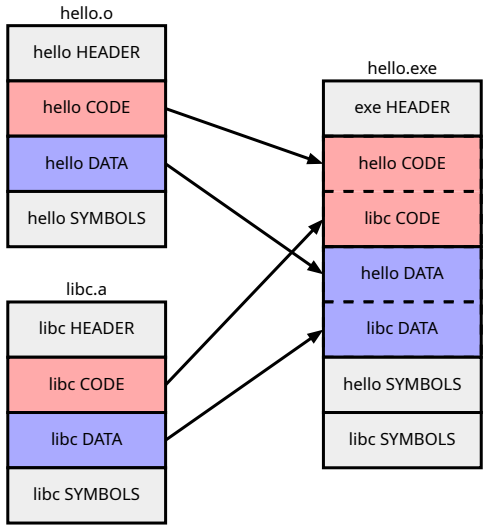
[1] Block Started by Symbol: Speicherbereich (normalierweise) mit Null initialisiert wird

Binden (*linking*)

- Binder kombiniert mehrere Objekt-Dateien und Bibliotheken
  - entweder zu ausführbare Datei (*executable*)
    - fügt automatisch Standardbibliothek (libc) hinzu
    - prüft ob alle Symbole definiert sind
  - oder zu Bibliothek (*library*)
    - prüft nicht ob alle Symbole definiert sind

```
gcc -c hello.c # -> hello.o
gcc -static -o hello -lc hello.o # -> hello(.exe)
./hello
```

```
Hello, World!
```



Beispiel Gnu Compiler Collection (GCC)

- Beispiel Übersetzen einer C-Datei in ausführbare Datei (*executable*)

```
gcc -std=c23 -pedantic \
    -Wall -Wextra -Wconversion -Wshadow \
    -o hello \
    hello.c
```

- `-o` Namen der resultierenden Datei
- `-std=c23` C23 Standard verwenden
  - `-pedantic` Warnung bei Nicht-Standard-Features
- `-W` Warnungen aktivieren
  - `-Wall` gängigste Warnungen
  - `-Wextra` zusätzliche Warnungen
  - `-Wconversion` fragwürdige Konvertierungen
  - `-Wshadow` Überschatten von Bezeichnern
- Zwischenschritt über Objektdatei wird im Hintergrund ausgeführt
  - direkte Übersetzung (`-c`) und Bindung (`-lc`) in einem Schritt